ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

NAME: DENISHA SHREE R

REG NO: 192524016

Course: Object Oriented Programming with C++ (DSA0101)

Course Outcome: CO4 – Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers and polymorphism to design reusable and extensible programs (constructors in derived classes, member classes, pointers to objects, the this pointer and pointers to derived classes)

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5

level) Mandatory Execution and GitHub Submission

• Every student must individually execute every task in this assignment, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment4 and share it with the course faculty (public, or private with the faculty added as a collaborator).

• Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.

• Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.

• Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.

• Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output

with the expected output.
Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to design a class hierarchy before writing its code, so that common behaviour is written once and new kinds of objects can be added without changing working classes.

You are expected to demonstrate the following progression:

Interpret the Requirement → Identify Is-a and Has-a Relationships → Design the Hierarchy → Decide Access → Implement → Trace Construction → Challenge → Refine

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. All code must compile with g++ -std=c++17 -Wall -Wextra.

TASK 1 – CLASS DESIGN REVIEW

Bloom's Level: K5 – Evaluate

Diagnose and Repair Duplicated Classes

6 Marks

A college payroll program has three separate classes written by different developers:

```
class Professor { public: string name; int id; double basic; double da; double research;
double pay() { return basic + basic * 0.42 + research; } };
class LabAssistant{ public: string name; int id; double basic; double
da;  double pay() { return basic + basic * 0.42; } };
class GuestFaculty{ public: string nm; int code; double hours; double rate;
double pay() { return hours * rate; } };
```

Task 1.1 – Diagnose the Design | 2 Marks

Identify at least five design deficiencies. Examine aspects such as:
• Repeated fields and repeated pay rules

• Public data members

• Inconsistent names for the same information

• The unused da field

• The magic number 0.42

• The effort needed to add a new staff category

• The inability to process all staff in one loop

Task 1.2 – Repair the Design | 3 Marks

Redesign the classes as a hierarchy with a common base class, protected or private data, and a pay() function overridden in each derived class. Draw the UML class diagram.

Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, explain how your hierarchy makes adding a new staff category easier.

Alternative Concept and Complexity Analysis

If the three staff types were represented by one Staff class with a type field and a switch inside pay(), instead of a base class with derived classes, what would the solution be? Analyse the time per pay() call (O(1) in both) and the space per object (unused fields in the single class), and count the places that must change when a new staff type is added.

Evidence Expected

A reasoned diagnosis, a corrected hierarchy with a UML diagram and a concise justification.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

Bloom's Level: K5 – Evaluate

University People Management

8 Marks

A university keeps records of persons (name, ID, phone). Students have a programme and CGPA; staff have a department and salary. A teaching assistant is both a student and a staff member. Research scholars are students who also hold a fellowship.

Task 2.1 – Identify the Relationships | 2 Marks

Classify each relationship as is-a (inheritance) or has-a (member class), and name the type of

inheritance (single, multiple, multilevel, hierarchical, hybrid) used for each part.

Task 2.2 – Resolve the Diamond | 2 Marks

Explain the problem that arises when TeachingAssistant inherits from both Student and Staff, and how virtual base classes solve it.

Task 2.3 – Design the Classes | 3 Marks

Write the class declarations with access specifiers, and decide for each base class whether it is inherited publicly, protectedly or privately.

Task 2.4 – Access Table | 1 Mark

Prepare a table showing how a public, protected and private member of Person is visible in Student under public, protected and private inheritance.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Person { Person() { std::cout << "Person "; } };

struct Student : virtual Person { Student() { std::cout << "Student "; }

}; struct Staff : virtual Person { Staff() { std::cout << "Staff "; } };

struct TeachingAssistant : Student, Staff {

 TeachingAssistant() { std::cout << "TA "; }

};


int main()

{

 TeachingAssistant ta;

 std::cout << '\n';

}
```

My predicted output (write before running):

Explain why Person is printed only once. Remove the word virtual from both base classes, predict the new output and compiler messages, and verify.

Alternative Concept and Complexity Analysis

If TeachingAssistant contained a Student object and a Staff object as members (composition, has-a), instead of inheriting from both through virtual base classes, what would the solution be? Analyse the space per object (one or two Person sub-objects) with sizeof, and compare the time and ambiguity of reaching the name field in each design.

Evidence Expected

A relationship analysis, a resolved diamond, class declarations and a correct access table.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER

CLASSES Bloom's Level: K5 – Evaluate

Order of Construction and Destruction

9 Marks

Task 3.1 – Pass Values to Base Constructors | 3 Marks

Write constructors for Student, Staff and TeachingAssistant that pass values to their base classes through initializer lists. Explain why the most-derived class must call the virtual base constructor directly.

Task 3.2 – Member Classes | 2 Marks

Add an Address member class to Person and a Course member class to Student. Show how their constructors receive values.

Task 3.3 – Trace the Order | 3 Marks

Print a message in every constructor and destructor and record the order for a Student, a Staff and a TeachingAssistant object. Explain the rule you observe.

Task 3.4 – Experiment | 1 Mark

Remove the virtual keyword from the base classes and record how the output and sizeof(TeachingAssistant) change.

Predict the Output
Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Engine {
 Engine() { std::cout << "Engine "; }
 ~Engine() { std::cout << "~Engine "; }
```

```
};

struct Vehicle {
 Vehicle() { std::cout << "Vehicle "; }
 ~Vehicle() { std::cout << "~Vehicle "; }
};

struct Car : Vehicle {
 Engine e;
 Car() { std::cout << "Car "; }
 ~Car() { std::cout << "~Car "; }
};

int main()
{
 {
 Car c;
 std::cout << "| ";
 }
 std::cout << '\n';
}
```

My predicted output (write before running):

State the rule for the order in which base classes, member objects and the derived class are constructed and destroyed.

Complete the Code

Complete TODO 1 below using a member initializer list, and execute the program. #include <iostream>

#include <string>

```
class Person {
protected:
 std::string name;
 int age;

public:
 Person(const std::string& n, int a) : name(n), age(a) {}
};

class Address { // used as a member class (composition)  std::string city;
 int pin;

public:
 Address(const std::string& c, int p) : city(c), pin(p) {}
 std::string str() const { return city + "-" + std::to_string(pin); }
};

class Student : public Person {
 long long regNo;
 Address home;

public:
 // TODO 1: Write the constructor: initialise the Person base, regNo and the home
// Address member in the initializer list
```

```
 void print() const
 {
 std::cout << regNo << ' ' << name << " (" << age << ") " << home.str() << '\n';  }
};

int main()
{
 Student s("Kavya", 19, 192311001, "Salem", 636001);
 s.print();
}
```

Expected output:

192311001 Kavya (19) Salem-636001

Swap the order of the items in your initializer list, compile with -Wall and record the warning.

Alternative Concept and Complexity Analysis
If Student held a pointer to a dynamically created Address (Address* home = new Address(…)) instead of an Address member object, what would the solution be? Analyse the extra time for allocation and the extra space for the pointer, and state what the constructor, copy constructor and destructor must now do.

Evidence Expected

Correct derived-class constructors, member classes, traced construction and destruction orders and an explained experiment.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

Bloom's Level: K5 – Evaluate

7 Marks

A bank stores different kinds of accounts through base-class pointers. Senior-citizen accounts

should earn 4.0% interest, but every account earns 3.5%, and memory reports show that part of each senior account is never released.

```
class Account { public: double rate() const { return 3.5; } ~Account() {}

}; class SeniorAccount : public Account {

 double* history = new double[100];

public: double rate() const { return 4.0; } ~SeniorAccount() { delete[] history; } };


Account* accounts[2] = { new Account, new SeniorAccount };

for (Account* a : accounts) std::cout << a->rate() << "\n";

for (Account* a : accounts) delete a;

Account copy = SeniorAccount(); // stored in a vector<Account> elsewhere
```

Task 4.1 – Locate the Defects | 2 Marks

Identify why the senior rate is never used, why memory leaks, and what object slicing does in the last line.

Task 4.2 – Construct Evidence | 2 Marks

Run the program under -fsanitize=address and add output to the destructors to prove each defect.

Task 4.3 – Repair the Program | 2 Marks
Correct the program using virtual functions, a virtual destructor and pointers (or references) instead of copies.

Task 4.4 – Prevent Recurrence | 1 Mark

State two rules that would have prevented these defects (for example: a base class with virtual functions needs a virtual destructor; use override).

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Account {

 double rate() const { return 3.5; }
```

```
 virtual double vrate() const { return 3.5; }
 virtual ~Account() = default;
};

struct SeniorAccount : Account {
 double rate() const { return 4.0; }
 double vrate() const override { return 4.0; }
};

int main()
{
 SeniorAccount   s;
 Account* p = &s;
 Account copy = s;
 std::cout << p->rate() << ' ' << p->vrate() << ' ' << copy.vrate() << '\n';
}
```

My predicted output (write before running):
Explain each of the three values: which call is bound at compile time, which at run time, and why the copy prints 3.5.

Alternative Concept and Complexity Analysis

If the accounts were stored as std::unique_ptr<Account> in a std::vector, instead of raw pointers deleted by hand, what would the solution be? Analyse whether time or space changes (the same, O(n) to process n accounts) and which of the three defects can still occur.

Evidence Expected

Correct defect identification, sanitizer and destructor evidence, a repaired program and prevention rules.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

Bloom's Level: K5 – Evaluate

7 Marks

A grocery delivery app keeps a shopping Cart object for each

customer. Task 5.1 – Four Ways to Reach an Object | 3 Marks

Write code that uses a Cart (a) directly by name, (b) through a pointer with ->, (c) through a reference, and (d) through a pointer created with new and released with delete. Use the this pointer to return *this so that calls can be chained, and to separate a member from a parameter with the same name.

Task 5.2 – Change the Requirement | 2 Marks

The app now keeps an array of pointers to Cart objects for all active customers. Write code that finds the cart with the highest total using pointer arithmetic or indexing.

Task 5.3 – Compare | 2 Marks

Compare the four ways on safety, lifetime and when each should be used. Explain why this is a pointer and what *this refers to.

Complete the Code

Complete TODO 1 and TODO 2 below using the this pointer, and execute the program.

```
#include <iomanip>

#include <iostream>

#include <string>


class Cart {

 std::string items[10];

 double prices[10];

 int n = 0;

 double discount = 0; // percent


public:

 // TODO 1: add(): store the item and price if there is space, and return *this so

// that calls can be chained




 // TODO 2: setDiscount(): use this-> to tell the member apart from the parameter

// named discount, and return *this
```

```
 double total() const
 {
 double t = 0;
 for (int i = 0; i < n; ++i) t += prices[i];
 return t * (100 - discount) / 100;
 }
};

int main()
{
 Cart c;
 c.add("Rice", 60).add("Dal", 120).setDiscount(10);
 Cart* p = &c; // pointer to an object
 p->add("Oil", 180);
 std::cout << std::fixed << std::setprecision(2) << p->total() << '\n';
}
```

Expected output:

324.00

Explain what would happen if add() returned Cart instead of Cart&.

Alternative Concept and Complexity Analysis

If the carts were kept in a std::vector<Cart> (objects) instead of an array of pointers to Cart objects, what would the solution be? Analyse the time to find the highest total (O(n) in both), the space (objects copied into the vector versus one pointer each) and what happens to pointers into the vector when it grows.

Evidence Expected

Working code for the four ways, method chaining with this, an array-of-pointers search and a reasoned comparison.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Vehicle Rental Fleet

13 Marks

A rental company rents bicycles, two-wheelers, cars and electric cars. All vehicles have a registration number and a daily rate. Motor vehicles also have a fuel or charge level and a service due date; cars have a seating capacity; electric cars have a battery capacity and are charged per kWh instead of per litre. Each vehicle computes its rental charge differently. The company needs to rent, return and list vehicles, and to add new vehicle types in future without changing the existing classes.

You are not given the design.

Stage 1 – Interpret the Problem | 2 Marks
Identify the entities, their data and the behaviour that differs between them. State the requirements as testable statements.

Stage 2 – Propose the Hierarchy | 2 Marks

Draw the class hierarchy, showing an abstract base class with at least one pure virtual function, the types of inheritance and any member classes.

Stage 3 – Identify the Critical Design Decisions | 2 Marks

Decide what is placed in each class, which functions are virtual, and how the fleet is stored (an array of base-class pointers).

Stage 4 – Design the Classes | 4 Marks

Present the class declarations and pseudocode for renting and returning a

vehicle. Stage 5 – Implement and Demonstrate | 3 Marks

Implement the program and demonstrate renting each type, returning with a late charge, and adding a new vehicle type (for example an electric bus) without changing the existing classes.

Alternative Concept and Complexity Analysis

If the fleet used a single Vehicle class with a type code and a switch in charge() instead of an abstract base class with virtual functions, what would the solution be? Analyse the time per charge() call (a switch versus a virtual call, both O(1)), the space per object (unused fields versus one vtable pointer), and the classes changed when a new vehicle type is added.

Evidence Expected

A complete hierarchy design with pseudocode, a working program and a demonstrated extension.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

• Report containing diagnoses, UML diagrams, access tables, construction traces, pseudocode and comparisons

• C++ source code for Tasks 1.2, 3, 4, 5 and 6

• Predictions, outputs and sanitizer reports

Assessment Verification

Students may be asked to explain any class, trace the construction of an object, or extend the hierarchy during a brief verification discussion/viva.

ANSWERS

ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Course: Object Oriented Programming with C++ (DSA01)

Name: _Narmatha J_

Register Number: _192521341_

Submission Date: _03-11-2026_

GitHub Repository Link: https://github.com/Narmatha5108/DSA0103--C-/tree/main/C%2B%2B%20Assignment%204/DSA01_Assignment4_CPP_Code

---

INTRODUCTION

This assignment explains how inheritance, virtual base classes, abstract classes, constructors, object pointers and polymorphism can be used to create reusable C++ programs.

Common information is stored in a suitable base class, while derived classes provide specialised behaviour.

The design process includes understanding the requirements, identifying class relationships, designing the hierarchy, implementing the classes, testing the programs and evaluating the results.

TASK 1 – CLASS DESIGN REVIEW (6 MARKS)

1.1 Diagnose the Design:

The original payroll classes have several design problems:

1. The classes Professor and LabAssistant repeat common data members such as name, ID and basic pay.
2. The calculation of basic pay plus 42% DA is repeated.
3. Public data members can be changed directly without proper validation.
4. Different variable names are used for the same information.
5. The DA field is declared but not properly used.

6. The value 0.42 is a magic number and should be declared as a constant.
7. Adding a new staff category requires creating more separate code.
8. The unrelated classes cannot be processed through one common Staff pointer.

1.2 Repair the Design:

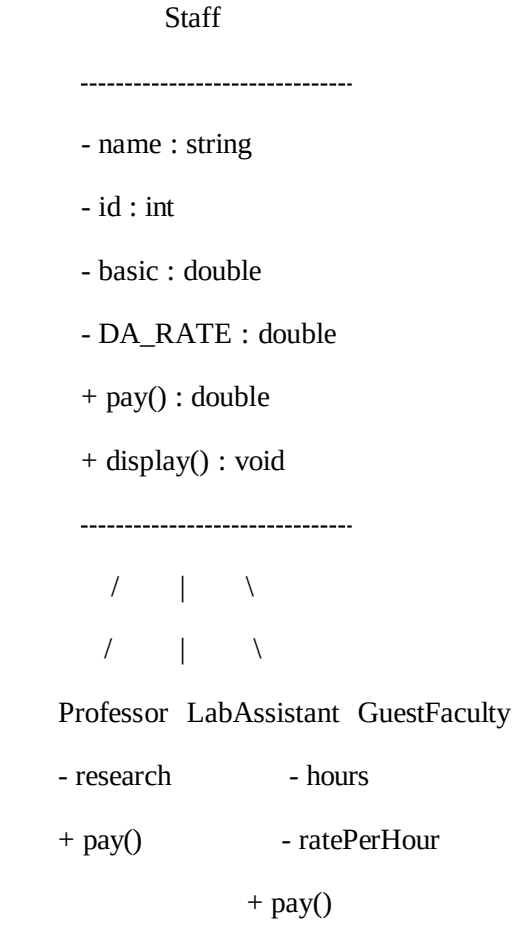
To solve these problems, an abstract base class named **Staff** is created. The Staff class stores common information such as name, ID and basic pay. The pay() function is declared as a pure virtual function because each staff category has a different salary calculation.

The following classes inherit from Staff:

- **Professor:** Receives basic pay, DA and research allowance.
- **LabAssistant:** Receives basic pay and DA.
- **GuestFaculty:** Receives payment based on working hours and hourly rate.

This design avoids repeated code and makes the program easier to maintain.

UML Class Diagram (Text Representation):



Public inheritance is suitable because Professor, LabAssistant and GuestFaculty are all types of Staff.

1.3 Design Defence:

The Staff base class stores information shared by all employees and provides a common pay() function. Each derived class implements its own salary calculation. A new employee category can be added by creating another class derived from Staff and overriding the pay() function. The existing payroll processing code does not need major changes. This reduces code duplication and improves reusability and maintenance.

Alternative Concept and Complexity Analysis:

An alternative design is to create a single Staff class with a type variable to identify Professor, LabAssistant or GuestFaculty.

The pay() function can use a switch statement to calculate the salary based on the employee type.

Both designs calculate one employee's salary in O(1) time.

However, the single-class design may contain unnecessary fields for some employees. For example, a LabAssistant does not need a research allowance field.

The inheritance approach keeps specialised information in the relevant derived class and makes it easier to add new employee categories.

Program File: Q1/Task01_PayrollHierarchy.cpp

TASK 2 – FROM REQUIREMENT TO HIERARCHY (8 MARKS)

2.1 Identify the Relationships:

The following classes are used:

- **Person:** Stores common personal information.
- **Student:** Inherits from Person.
- **Staff:** Inherits from Person.
- **TeachingAssistant:** Inherits from both Student and Staff.
- **ResearchScholar:** Inherits from Student.

The relationships are:

1. Student and Staff inherit from Person. This represents hierarchical inheritance.
2. TeachingAssistant inherits from both Student and Staff. This represents multiple inheritance.

3. ResearchScholar inherits from Student. This represents multilevel inheritance.

Since the design combines different types of inheritance, it is called **hybrid inheritance**.

2.2 Resolve the Diamond Problem:

If Student and Staff both inherit normally from Person, TeachingAssistant receives two copies of the Person class.

One Person object comes through Student, and another Person object comes through Staff.

This creates the diamond problem. Accessing a common member such as name may become ambiguous.

To solve this problem, virtual inheritance is used.

```
class Student : virtual public Person

{

};

class Staff : virtual public Person

{

};
```

Virtual inheritance ensures that TeachingAssistant shares only one Person base-class object.

2.3 Design the Classes:

The class relationships are:

```
class Student : virtual public Person

{

};

class Staff : virtual public Person

{

};

class TeachingAssistant : public Student, public Staff

{

};

class ResearchScholar : public Student
```

{

};

The most-derived class, TeachingAssistant, directly initialises the virtual Person base class through its constructor.

ResearchScholar also initialises Person because it is the most-derived class in that inheritance path.

2.4 Access Specifier Table:

Inheritance Type	Public Members	Protected Members	Private Members
Public	Public	Protected	Not directly accessible
Protected	Protected	Protected	Not directly accessible
Private	Private	Private	Not directly accessible

Private members of the base class cannot be directly accessed by derived classes. They can be accessed through suitable public or protected member functions.

Predict the Output – Virtual Diamond

Predicted Output:

Person Student Staff TA

Explanation:

Person is printed only once because Student and Staff use virtual inheritance.

The construction order is:

1. Person is constructed first.
2. Student is constructed.
3. Staff is constructed.
4. TeachingAssistant is constructed.

The virtual base class is constructed once by the most-derived class.

If virtual inheritance is removed, the usual output becomes:

Person Student Person Staff TA

In this case, two Person objects are created.

Alternative Concept and Complexity Analysis:

- Another method is composition, where TeachingAssistant contains separate Student and Staff objects.
- However, each contained object may have its own Person information. This can result in duplicated personal data.
- Virtual inheritance provides one shared Person object, while composition keeps separate contained objects.
- Accessing a member takes O(1) time in either design. The exact object size depends on the compiler and platform.

Program Files:

- Q2/Task02_PeopleHierarchy.cpp
- Q2/PredictOutput_VirtualDiamond.cpp

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES (9 MARKS)

3.1 Pass Values to Base Constructors:

A derived class can pass values to its base class using a constructor initializer list.

Example:

```
Student(const string& n, int a, long long r)
   : Person(n, a), regNo(r)
{
}
```

Here, the Student constructor passes the name and age to the Person constructor.

The base-class constructor is executed before the derived-class constructor body.

When virtual inheritance is used, the most-derived class is responsible for initialising the virtual base class.

3.2 Member Classes:

A class can contain objects of another class as its data members.

This relationship is called composition.

Examples:

- Person has an Address.
- Student has a Course.

Example:

Address home;

Course course;

The Address and Course objects can be initialised through the constructor initializer list.

This makes the program easier to organise and maintain.

3.3 Trace the Construction and Destruction Order:

For the given Car program, the predicted output is:

Vehicle Engine Car | ~Car ~Engine ~Vehicle

Construction Order:

1. Vehicle base class is constructed.
2. Engine member object is constructed.
3. Car constructor body is executed.

Destruction Order:

1. Car destructor is executed.
2. Engine member object is destroyed.
3. Vehicle base class is destroyed.

Destruction takes place in the reverse order of construction.

For the Student program:

1. Person base class is constructed.
2. Address member inside Person is constructed.
3. Course member inside Student is constructed.
4. Student constructor body is executed.

During destruction, the order is reversed.

3.4 Experiment:

When virtual inheritance is removed from a diamond hierarchy, TeachingAssistant contains two Person base-class objects instead of one. The constructor output contains two Person messages.

The size of TeachingAssistant may increase, but the exact value depends on the compiler, platform and memory alignment.

The actual value should be obtained by executing sizeof(TeachingAssistant) on the system used for testing.

Complete the Code – TODO 1

Constructor:

```
Student(const std::string& n, int a, long long r,
        const std::string& city, int pin)
    : Person(n, a), regNo(r), home(city, pin)
{
}
```

Expected Output:

```
192311001 Kavya (19) Salem-636001
```

The order of construction depends on the order in which data members are declared in the class, not the order in which they appear in the initializer list.

Alternative Concept and Complexity Analysis:

- Address can also be stored using a pointer.
- However, dynamic memory allocation requires the constructor to allocate memory and the destructor to release it.
- A correct copy constructor and copy-assignment operator are also required to avoid problems such as double deletion.
- Using a direct member object is simpler because C++ automatically manages its lifetime.

Program Files:

- Q3/Task03_ConstructorsAndMembers.cpp
- Q3/CompleteCode_AddressMember.cpp

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS (7 MARKS)

4.1 Locate the Defects:

The original program contains the following problems:

1. Account::rate() is not virtual. Therefore, calling it through an Account pointer uses the base-class version.
2. Account does not have a virtual destructor. Deleting a derived object through a base-

class pointer can cause undefined behaviour.

3. SeniorAccount dynamically allocates memory using new[]. The allocated memory must be released using delete[].
4. Copying a SeniorAccount object into an Account object causes object slicing.
5. Object slicing removes the derived-class-specific information.

4.2 Construct Evidence:

To observe the constructor and destructor behaviour, messages can be added to both classes.

The original defective program can be compiled using AddressSanitizer:

g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -fno-omit-frame-pointer original.cpp -o original

The program should then be executed and the actual terminal output should be saved.

The exact AddressSanitizer message depends on the compiler and runtime environment.

4.3 Repair the Program:

The following changes solve the problems:

1. Declare rate() as a virtual function in Account.
2. Use override in SeniorAccount.
3. Declare the Account destructor as virtual.
4. Use pointers or references to preserve derived-class objects.
5. Use smart pointers such as unique_ptr to manage dynamic memory safely.

Example:

```
class Account
{
public:
   virtual double rate() const
   {
      return 3.5;
   }
```

```
    virtual ~Account() = default;
};
```

The virtual destructor ensures that the derived-class destructor is called correctly when an object is destroyed through a base-class pointer.

4.4 Prevent Recurrence:

The following practices help prevent similar problems:

1. A base class used polymorphically should have a virtual destructor.
2. Use override in derived classes to check correct function overriding.
3. Avoid storing derived objects by value in a base-class container because it causes object slicing.
4. Use smart pointers for safe memory management.

Predict the Output:

Predicted Output:

3.5 4 3.5

Explanation:

- p->rate() calls the non-virtual Account::rate() function and prints 3.5.
- p->vrate() calls the virtual function and selects SeniorAccount::vrate(), printing 4.
- copy.vrate() prints 3.5 because the derived object has been copied into a base-class object. This is called object slicing.

Alternative Concept and Complexity Analysis

- Using vector<unique_ptr<Account>> allows different derived objects to be stored and processed through a common base-class pointer.
- Processing n accounts takes O(n) time.
- Each rate() function call takes O(1) time.
- Smart pointers automatically manage object ownership and reduce the risk of memory leaks.

Program Files:

- Q4/Task04_VirtualFunctionsAndSlicing.cpp
- Q4/Task04_RepairedAccounts.cpp

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER (7 MARKS)

5.1 Four Ways to Access an Object:

There are four common ways to access an object in C++.

1. Direct Object:

```
Cart c;
c.add("Rice", 60);
```

The object is accessed directly using its variable name.

2. Pointer to an Existing Object:

```
Cart* p = &c;
p->add("Dal", 120);
```

A pointer stores the address of an existing object. The arrow operator is used to access its members.

3. Reference:

```
Cart& ref = c;
ref.add("Oil", 180);
```

A reference acts as another name for the same object.

4. Dynamic Object:

```
Cart* p = new Cart;
p->add("Book", 100);
delete p;
```

The object is created dynamically using new and released using delete.

The this Pointer:

The this pointer refers to the current object.

Inside a member function:

- this represents the address of the current object.
- *this represents the current object itself.
- this->discount refers to the data member named discount.

Returning *this by reference allows method chaining.

5.2 Find the Highest Total in an Array of Pointers

Pseudocode:

```
SET highestCart TO carts[0]

FOR i FROM 1 TO count - 1

   IF carts[i]->total() > highestCart->total()

      SET highestCart TO carts[i]

   END IF

END FOR

DISPLAY  highestCart->total()
```

The array stores pointers to Cart objects.

The arrow operator is used to access the total() function of each object.

5.3 Compare the Four Methods:

Method	Description
Direct object	Accessed using the object name
Pointer	Stores the address of an object
Reference	Acts as another name for an object
Dynamic object	Created using new and released using delete

Direct object access is simple and safe while the object is in scope.

Pointers can be reassigned and may contain null values, so they must be checked before use.

References are aliases for existing objects.

Dynamic objects can have a longer lifetime, but their memory must be managed correctly.

Complete the Code – TODO 1:

```
Cart& add(const std::string& item, double price)

{

   if (n < 10 && price >= 0)

   {

      items[n] = item;
```

```
        prices[n] = price;
        ++n;
    }
    return *this;
}
```

Complete the Code – TODO 2:

```
Cart& setDiscount(double discount)
{
    if (discount >= 0 && discount <= 100)
    {
        this->discount = discount;
    }
    return *this;
}
```

Expected Output:

```
324.00
```

Returning Cart& allows the same object to be used in a chain of function calls.

If the function returns Cart by value, a copy of the object is returned instead.

Alternative Concept and Complexity Analysis:

- Instead of storing pointers, Cart objects can be stored directly in a vector<Cart>.
- Finding the highest total takes O(n) time because every cart must be checked.
- A vector of objects avoids separate pointer ownership, while an array of pointers allows objects to be stored separately.
- However, pointers and references to vector elements may become invalid if the vector reallocates its storage.

Program Files:

- Q5/Task05_CartThisPointer.cpp
- Q5/CompleteCode_ThisPointer.cpp

TASK 6 – DESIGN CHALLENGE: VEHICLE RENTAL FLEET (13 MARKS)

Stage 1 – Interpret the Problem:

The objective is to design a vehicle rental management system using inheritance and polymorphism.

The system manages different types of vehicles and calculates their rental charges.

The main entities are:

1. Vehicle
2. Bicycle
3. MotorVehicle
4. TwoWheeler
5. Car
6. ElectricCar
7. ElectricBus

The Vehicle class stores common details such as registration number, daily rental rate and rental status.

MotorVehicle contains additional details such as fuel or charge level and service due date.

Car contains seating capacity.

ElectricCar contains battery capacity and price per kWh.

Stage 2 – Propose the Hierarchy:

The proposed class hierarchy is:

Vehicle (Abstract)

- registration

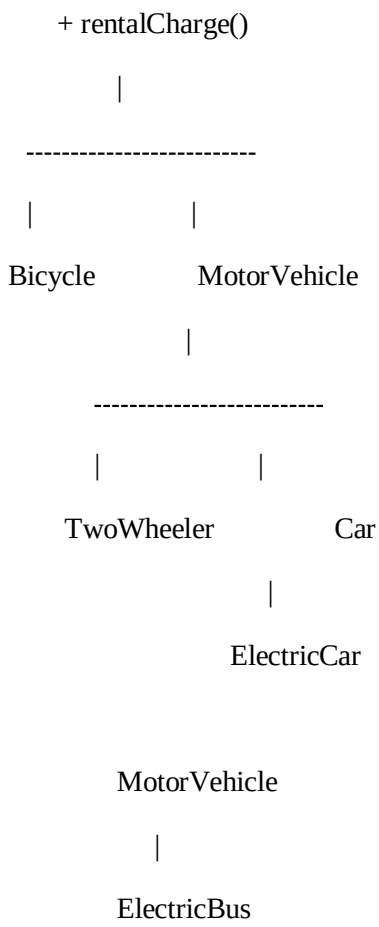
- dailyRate

- rented

+ rent()

+ returnVehicle()

+ showDetails()



Inheritance represents an is-a relationship.

For example:

- A Bicycle is a Vehicle.
- A Car is a MotorVehicle.
- An ElectricCar is a Car.
- An ElectricBus is a MotorVehicle.

Stage 3 – Critical Design Decisions:

The Vehicle class is declared as an abstract class because each vehicle type calculates its rental charge differently.

The rentalCharge() function is declared as a pure virtual function.

The following functions are used:

- rent(): Marks a vehicle as rented.
- returnVehicle(): Marks a vehicle as available and calculates late charges.

- showDetails(): Displays vehicle information.
- rentalCharge(): Calculates the rental amount.
- type(): Returns the vehicle category.

The Vehicle destructor is declared as virtual to support safe destruction through base-class pointers.

Common data is stored in the Vehicle class, while specialised data is stored in the relevant derived classes.

Stage 4 – Design the Classes and Pseudocode:

Rent Vehicle Pseudocode:

```
BEGIN

INPUT registration number

SEARCH for  the vehicle

IF vehicle is not found

   DISPLAY "Vehicle not found"

ELSE IF vehicle is already rented

   DISPLAY "Vehicle already rented"

ELSE

   MARK vehicle as rented

   DISPLAY "Vehicle rented successfully"

END IF

END
```

Return Vehicle Pseudocode:

```
BEGIN

INPUT registration number and late days

SEARCH for the vehicle

IF vehicle is not found OR vehicle is not rented

   DISPLAY an error message
```

ELSE

CALCULATE late charge

DISPLAY return details

MARK vehicle as available

END IF

END

Stage 5 – Implementation and Demonstration:

The program creates five types of vehicles:

1. Bicycle
2. TwoWheeler
3. Car
4. ElectricCar
5. ElectricBus

Their addresses are stored in an array of Vehicle pointers.

A single loop displays the vehicle details and calculates the rental charge for two days.

The program also demonstrates renting and returning vehicles.

The Car is rented and returned with two late days. The ElectricBus is also rented and returned.

The ElectricBus is added as a new derived class without changing the existing Vehicle interface.

This demonstrates the advantages of inheritance and polymorphism.

Alternative Concept and Complexity Analysis:

An alternative design is to create a single Vehicle class with a type code and use a switch statement to calculate the rental charge.

Both switch-based calculation and virtual function calls take O(1) time for one vehicle.

Processing n vehicles takes O(n) time.

However, a single-class design may contain unnecessary data members for some vehicle types.

For example, a Bicycle does not require battery capacity or seating capacity.

Inheritance keeps specialised information inside the relevant derived classes.

It also makes it easier to add new vehicle categories without modifying the existing classes.

Program File: Q6/Task06_VehicleRentalFleet.cpp

COMPILATION AND EXECUTION

All programs should be compiled using a C++17-compatible compiler.

Compilation Command:

```
g++ -std=c++17 -Wall -Wextra -pedantic filename.cpp -o program
```

Execution Command in Windows PowerShell:

```
.\program.exe
```

Each program should be compiled and executed separately.

The actual terminal output should be captured and saved as a screenshot inside the matching task folder.

For Task 4, the original defective program can also be tested using AddressSanitizer. The actual output from the student's system should be included.

CONCLUSION

This assignment demonstrates how inheritance and polymorphism help create reusable C++ class hierarchies.

Virtual inheritance solves the diamond problem, while constructors and destructors control object creation and destruction.

Virtual functions support runtime polymorphism, and object pointers allow different derived objects to be processed through a common interface.

The this pointer supports method chaining.

The vehicle rental system demonstrates how an abstract base class can be extended to support different vehicle types without changing the existing common interface.

Final Result:

The C++ programs are designed to demonstrate inheritance, virtual inheritance, constructors, destructors, virtual functions, object pointers and polymorphism through practical examples.